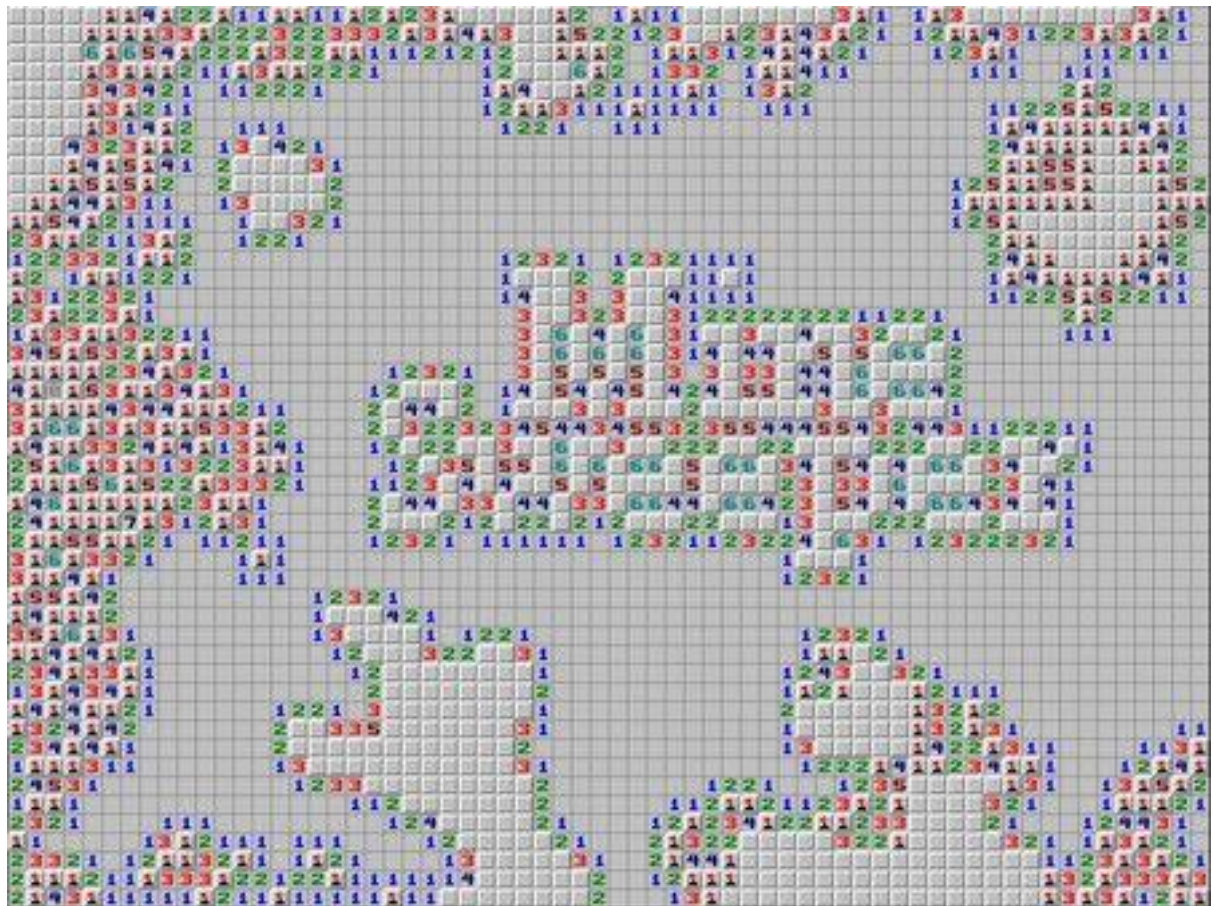


## Documentación técnica buscaminas



## Índice

|                                        |   |
|----------------------------------------|---|
| Documentación técnica buscaminas ..... | 1 |
| Diseño del software .....              | 3 |
| Diseño de clases .....                 | 3 |
| Diagrama de flujo .....                | 4 |
| Arquitectura del proyecto.....         | 5 |
| Diseño de la base de datos.....        | 6 |
| Modelo E-R .....                       | 6 |
| Diseño físico .....                    | 6 |
| Diseño web .....                       | 7 |
| Wireframes .....                       | 7 |
| Experiencia de usuario .....           | 7 |

## Diseño del software

### Diseño de clases

El software se divide en tres responsabilidades principales: la gestión del estado individual de cada celda, la lógica del tablero y las reglas del juego, y el controlador principal que une todo.

#### 1. Clase Celda(Cell)

Representa la unidad mínima del juego. Su responsabilidad es conocer su propio estado, pero no sabe sobre sus vecinas ni sobre el juego global.

- **Atributos:** fila(int), columna(int), tieneMina(boolean), estaDescubierta(boolean), estaMarcada(boolean), minasVecinas(int).
- **Métodos:** descubrir(), marcar(), esMina(), obtenerEstado().
- **Responsabilidad:** Encapsular la información de una única posición. No calcula cuántas minas tiene alrededor; solo almacena el resultado cuando el tablero se lo indica.

#### 2. Clase Tablero(Board / Field)

Es el núcleo de la lógica. Gestiona la matriz de celdas, coloca las minas y calcula los números de seguridad.

- **Atributos:** filas(int), columnas(int), totalMinas(int), matrizCeldas(Lista de Listas de Celda), juegoTerminado(boolean).
- **Métodos:** inicializar(), colocarMinasAleatorias(), calcularMinasVecinas(), revelarCelda(fila, columna), verificarVictoria().
- **Lógica Clave:** El método revelarCelda() debe implementar la **recursividad**: si se descubre una celda con 0 minas vecinas, el tablero debe llamar automáticamente a revelarCelda() para todas sus vecinas, descubriendo áreas vacías en cascada.

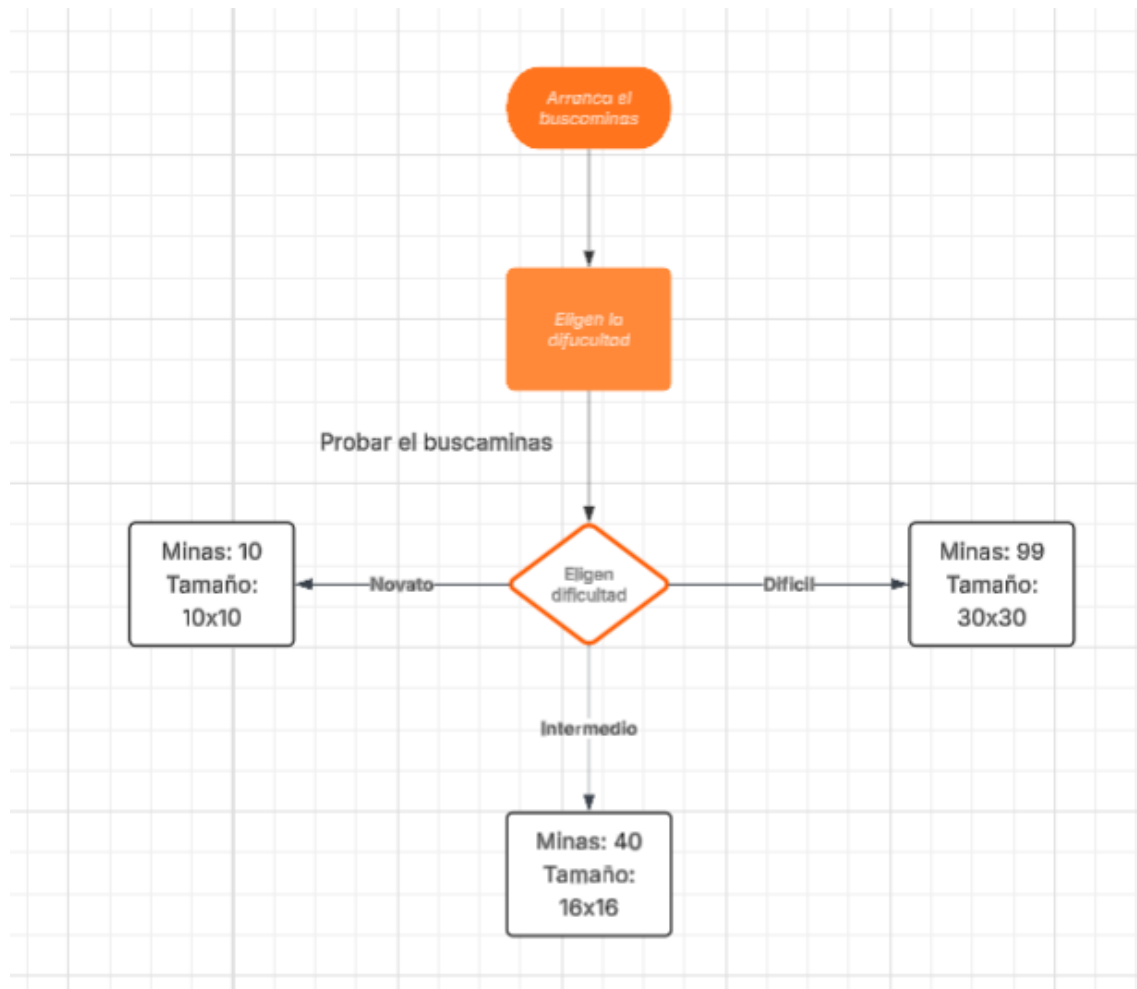
#### 3. Clase Juego(Game Controller)

Actúa como el orquestador. Controla el flujo, el temporizador y el estado global (Ganado/Perdido).

- **Atributos:** tablero(Objeto Tablero), tiempoTranscurrido(int), estado(Enum: Jugando, Ganado, Perdido).
- **Métodos:** iniciarJuego(), manejarClick(fila, columna), detenerJuego(), reiniciar().

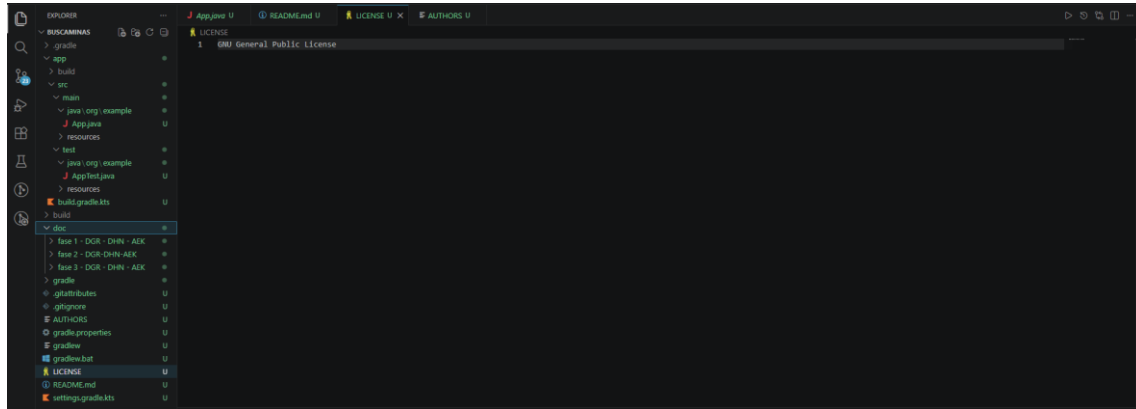
- **Responsabilidad:** Recibe la entrada del usuario (o de la interfaz), se la pasa al tablero y verifica si la partida ha terminado tras cada movimiento.

### Diagrama de flujo



## Arquitectura del proyecto

Utilizaremos Visual Studio Code y Netbeans, usaremos el visual studio para realizar la arquitectura, para luego realizar el build y mas adelante para comprobar errores, mientras que usaremos Netbeans para realizar los test, la programación del Buscaminas y la parte visual con javafx



## Diseño de la base de datos

### Modelo E-R

En nuestra base de datos donde guardemos en la base de datos el **juego**, el **jugador** y un **ranking**.

En el juego queremos guardar en que dificultad están jugando y el tiempo que han estado jugando durante el juego hasta que haya ganado o haya perdido.

En el jugador queremos guardar el ID del jugador y su nombre o el apodo con el que entren.

En el ranking este la puntuación (que será una multiplicación entre la dificultad, el tiempo y si ha ganado el juego o perdido, y en caso de haber perdido cuanto ha avanzado en el buscaminas) junto a la dificultad (ya que las puntuaciones que se hagan en una dificultad estarán dentro de donde tienes las mismas dificultades) y un id (el cual será las tres primeras letras del nombre/apodo del jugador)

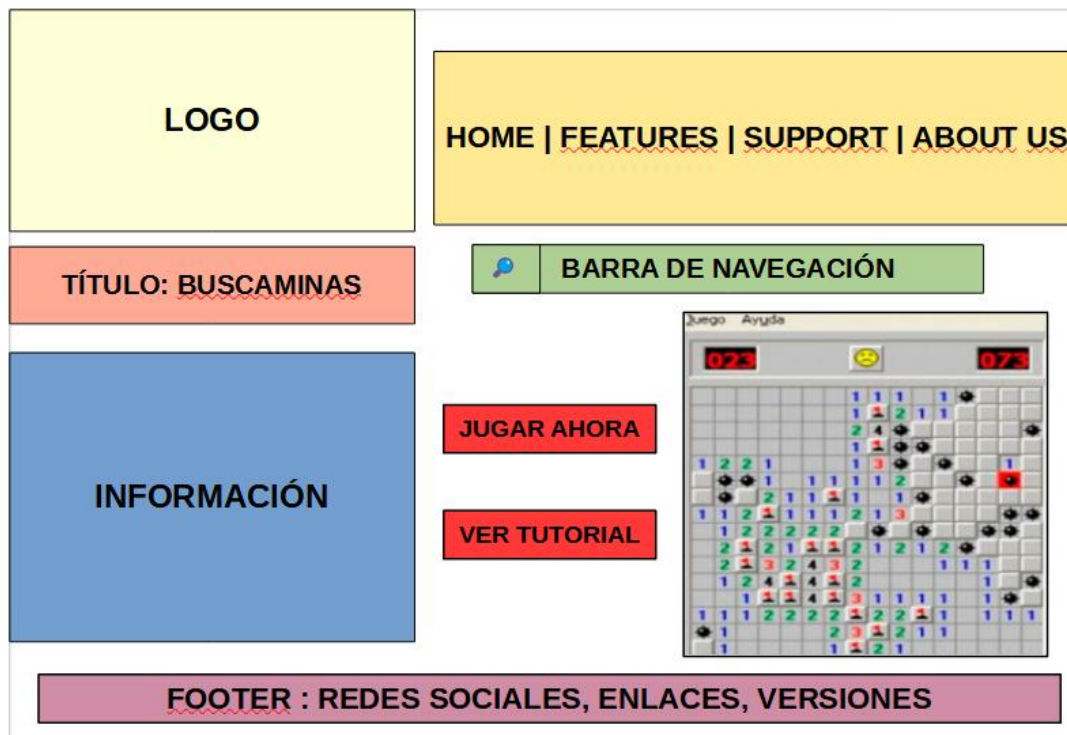
### Diseño físico

| Taula                            | Acció                                            | Fila     | Tipus         | Col·lació                    | Mida           | Desfragmentat |
|----------------------------------|--------------------------------------------------|----------|---------------|------------------------------|----------------|---------------|
| <input type="checkbox"/> juego   | ★ Navega Estructura Cerca Insereix Buida Elimina | 0        | InnoDB        | utf8mb4_uca1400_ai_ci        | 32.0 KB        | -             |
| <input type="checkbox"/> jugador | ★ Navega Estructura Cerca Insereix Buida Elimina | 0        | InnoDB        | utf8mb4_uca1400_ai_ci        | 16.0 KB        | -             |
| <input type="checkbox"/> ranking | ★ Navega Estructura Cerca Insereix Buida Elimina | 0        | InnoDB        | utf8mb4_uca1400_ai_ci        | 48.0 KB        | -             |
| <b>3 taules</b>                  | <b>Suma</b>                                      | <b>0</b> | <b>InnoDB</b> | <b>utf8mb4_uca1400_ai_ci</b> | <b>96.0 KB</b> | <b>0 B</b>    |

☐ Marca-ho tot

## Diseño web

### Wireframes



### Experiencia de usuario

Utilizando el enlace de la descarga del buscaminas eliminaremos la necesidad de realizar un documento de instalación como sucede con varios programas los cuales se tienen que descargar de un enlace, y al pesar tan poco eliminamos el riesgo de que nuestra aplicación suponga un riesgo a la memoria de nuestros usuarios/clientes, además es una instalación bastante sencilla lo cual minimiza el riesgo de errores o equivocaciones a la hora de instalar el buscaminas